

Amazon DynamoDB, carte mentale

PIÈGE classique à l'examen **PRÉCISION** corrigé ou complète tes notes Sigles soulignés en pointillé : voir le glossaire en bas

DynamoDB

NoSQL clé-valeur et document

01 Fondamentaux

- Base **NoSQL** **clé-valeur** et **document**, entièrement gérée et **serverless** (aucun serveur à gérer)
- Scale **horizontalement** et automatiquement
- Supporte **ACID** grâce aux transactions
- Facturation : **on-demand** (par requête) ou **provisioned** (capacité réservée)
- ❖ **PIÈGE** Pas de jointures : les données sont **dénormalisées**, dupliquées volontairement
- Côté AWS : RDS pour le relationnel ; DynamoDB, ElastiCache, Neptune pour le non relationnel

	RELATIONNELLE	NOSQL
Exemples	SQL Server, Oracle	Redis, MongoDB, DynamoDB
Schéma	Strict, défini d'avance	Flexible, dynamique
Scaling	Vertical (serveur plus gros)	Horizontal (plus de nœuds)
Choisir si	Schéma strict, requêtes complexes, pas de perf R/W extrême	Scale horizontal, paires clé-valeur, schéma dynamique, très gros débit

02 Composants et types de données

- Table** = collection d'items (= table SQL, sans schéma fixe)
- Item** = un enregistrement (= ligne), un ensemble d'attributs
- Attribut** = un champ (= colonne)
 - Un item peut avoir des attributs qu'un autre item de la même table n'a pas
- Seule la clé primaire est obligatoire
- ❖ **PIÈGE** Taille max d'un item : **400 KB** (noms + valeurs d'attributs)
- Scalaire** (une seule valeur) : String, Number, Binary, Boolean, Null
- Sets** (plusieurs valeurs scalaires du même type) : String Set, Number Set, Binary Set
- Documents** (structures imbriquées type JSON) : List, Map

03 Clés primaires et partitionnement

- Clé simple** : **partition key** seule (hash attribute), unique pour chaque item
- Clé composite** : partition key + **sort key**
 - La partition key seule peut se répéter
 - La combinaison PK + SK doit être unique
- Pas d'auto-incrément : on utilise un **UUID** ou un **ID** applicatif
- La **valeur hashée** de la PK décide de la partition (allocation de stockage)
- Items de même PK stockés **ensemble**, triés par SK
 - Un **Query** sur une PK lit un bloc contigu : c'est ce qui le rend rapide
- ❖ **PIÈGE** **Hot partition** : une PK qui reçoit trop de trafic limite tout, même si la table est sous sa capacité
 - Bonne PK = forte cardinalité, trafic réparti
- ❖ **PRÉCISION** « Chaque valeur de PK a sa propre partition » est une **spécification** : plusieurs valeurs peuvent partager une partition physique (limite ≈ 10 GB, 3 000 RCU, 1 000 WCU par partition)

04 Index secondaires

chercher autrement que par la clé primaire

POURQUOI

- Sans index, on ne cherche que par la clé primaire
- Un index définit une **clé alternative** pour interroger sur un autre attribut
- Pour l'utiliser : **Query** avec `IndexName`

GSI CONTRE LSI

	GSI	LSI
Portée	Au-delà de la partition (toutes les partitions)	Dans la même partition
Clés	PK et SK différentes de la table	Même PK, SK différente
Débit	RCU/WCU propres	Partagé avec la table
Taille	Pas de limite	10 GB par valeur de PK
Création	À tout moment	Seulement à la création de la table
Maximum	20 par table	5 par table
Lecture	Eventually consistant seulement	Supporte strongly consistant

Un LSI enferme les données d'une PK dans une seule partition (10 GB max). Si ça peut déborder, il faut prévoir un GSI dès le design.

PROJECTION : QUOI COPIER DANS L'INDEX

- KEYS_ONLY** : seulement les clés
- INCLUDE** : clés + attributs choisis
- ALL** : tous les attributs
- Bonne pratique : projeter le minimum (moins de coût et de latence)

CHOISIR LA CLÉ D'UN GSI

- Un GSI regroupe des items venant de partitions différentes. Exemple : `status` (LEGIT / FRAUD / PENDING) permet « tous les items avec status = X, tous users confondus », impossible avec la PK/SK de la table

- La SK du GSI (ex. `created_at`) trie chaque groupe. Une SK qui varie par item est normale

- ❖ **PRÉCISION** Une PK de GSI à très peu de valeurs concentre les écritures et peut créer une **hot partition**
 - Un GSI en throttling fait aussi throttler les écritures de la table de base
 - Parade : ajouter un suffixe de sharding ou combiner la valeur avec une date

SPARSE INDEX

- Seuls les items qui **possèdent** l'attribut clé de l'index y apparaissent
- Utile pour un index léger : ne remplir l'attribut que quand `status = PAID`

05 Opérations de l'API

LES 4 CATÉGORIES

- Contrôle** : créer et gérer les tables (`CreateTable` : nom, schéma de clés, définition des attributs, débit provisionné)
- Données** (CRUD) : `PutItem`, `GetItem`, `UpdateItem`, `DeleteItem`, `Query`, `Scan`
- Batch** : `BatchGetItem`, `BatchWriteItem`
- Transaction** : `TransactWriteItems`, `TransactGetItems`, changements coordonnés, tout ou rien, sur une ou plusieurs tables

LIRE

OPÉRATION	CLÉ REQUISE	RÔLE
<code>GetItem</code>	Clé primaire complète (PK + SK si composite)	Un seul item. Eventually consistant par défaut
<code>Query</code>	PK obligatoire (+ condition sur la SK)	Tous les items d'une PK. Plus efficace que Scan. Table ou index
<code>Scan</code>	Aucune	Parcourt toute la table ou l'index. Coûteux, à éviter en prod

ÉCRIRE

- ❖ `DeleteItem` : supprime un item par sa clé primaire
- Condition Expressions** : s'appliquent aux écritures, l'opération n'a lieu que si la condition est vraie
- Ajouter un attribut plus tard à un item existant : `UpdateItem`, sans migration

06 Cohérence de lecture et transactions

COHÉRENCE

- Les données sont copiées sur plusieurs serveurs : la synchronisation prend environ **1 s**, d'où un possible décalage à la lecture
- Eventually consistent** (défaut) : peut renvoyer une valeur périmée, **0,5 RCU** par 4 KB, moins cher
- Strongly consistent** : reflète la dernière écriture confirmée, **1 RCU** par 4 KB, plus cher
 - Paramètre `ConsistentRead = true`
- ❖ **PIÈGE** Un **GSI** ne fait que de l'eventually consistent. Un **LSI** et la table peuvent faire du strongly

TRANSACTIONS ACID

- ❖ `TransactWriteItems` : Put / Update / Delete, avec éventuelles conditions, en un bloc tout ou rien
- ❖ `TransactGetItems` : plusieurs lectures cohérentes ensemble
- Si une opération ou une condition échoue, **toute la transaction est rejetée**, rien n'est appliqué à moitié
- Coût doublé par rapport à une opération normale (voir branche 13)

07 Débit et capacité

RCU, WCU et choix du mode

LES 2 MODES

- Provisioned** : tu fixes les RCU et WCU à l'avance. Moins cher pour un trafic stable, élevé et prévisible
- On-demand** : DynamoDB s'adapte tout seul, facturé à la requête. Idéal pour un trafic imprévisible ou en pics, ou sans envie de planifier
 - Les unités s'appellent **RRU** / **WRU**, mêmes tailles (4 KB / 1 KB)

DÉFINITIONS

- RCU** = 1 lecture **strongly** consistant par seconde pour **4 KB** (ou 2 lectures eventually)
- WCU** = 1 écriture par seconde pour **1 KB**
- 3 infos pour calculer : nombre d'opérations par seconde, taille moyenne des items, type de cohérence

FORMULES

$$\begin{aligned} \text{RCU strongly} &= \text{arrondi}!(\text{taille} \div 4 \text{ KB}) \times \text{lectures/s} \\ \text{RCU eventually} &= \text{arrondi}!(\text{taille} \div 4 \text{ KB}) \times \text{lectures/s} \div 2 \\ \text{WCU} &= \text{arrondi}!(\text{taille} \div 1 \text{ KB}) \times \text{écritures/s} \end{aligned}$$

- ❖ **PIÈGE** On arrondit **toujours la taille au supérieur** avant de multiplier

EXEMPLES

CAS	CALCUL	RÉSULTAT
4 KB, strongly, 30 lectures/s	1×30	30 RCU
4 KB, eventually, 30 lectures/s	$1 \times 30 \div 2$	15 RCU
6 KB, strongly, 10 lectures/s	2×10	20 RCU
8 KB, strongly, 1 lecture/s	2×1	2 RCU
1 KB, 14 écritures/s	1×14	14 WCU
2,5 KB, 10 écritures/s	3×10	30 WCU

EN PRODUCTION

- Démarrer en **on-demand**
- Observer environ **2 semaines** dans CloudWatch : RCU/s, WCU/s, taille des items
- Si le trafic est prévisible, passer en **provisioned**
- Demande qui explose → erreur **400** (`ProvisionedThroughputExceededException`) → **Auto Scaling** ou retour en on-demand

08 Streams

- Flux, dans l'ordre chronologique, de toutes les modifications d'une table : **insert**, **update**, **delete**
- ❖ **PIÈGE** Conservé **24 h** seulement
- Option à activer sur la table
- Chaque modification produit un **stream record** : le changement, qu'une application peut lire pour réagir
 - Type d'événement : `INSERT`, `MODIFY`, `REMOVE`
 - Un `SequenceNumber` garde l'ordre
 - Les records sont regroupés en **shards**

LES 4 TYPES DE VUE

TYPE	CONTENU DU RECORD
<code>KEYS_ONLY</code>	Les clés de l'item modifié
<code>NEW_IMAGE</code>	L'item après modification
<code>OLD_IMAGE</code>	L'item avant modification
<code>NEW_AND_OLD_IMAGES</code>	Les deux

USAGES

- Déclencher une **Lambda** à chaque écriture (event-driven)
- Réplication (base des Global Tables)
- Audit, synchronisation entre systèmes, Kinesis

09 Global Tables

- Table répliquée **automatiquement** dans plusieurs régions AWS, une réplique par région, copie identique, via les **Streams**
- Base **multi-région** et **multi-active** : lecture et écriture dans chaque région, sans construire sa propre répllication
- On choisit les régions où la table doit être disponible
- Mise à jour propagée en quelques secondes
- Conflit (même item écrit dans 2 régions) : **last writer wins**, selon l'horodatage
- ❖ **PIÈGE** Lecture **strongly consistent** seulement dans la région où l'écriture a eu lieu ; entre régions, c'est eventually
- ❖ **PRÉCISION** Les transactions ACID ne valent que **dans la région** où elles s'exécutent, pas d'une région à l'autre
- ❖ **PRÉCISION** Depuis 2015, mode optionnel **MRSC** (*multi-Region strong consistency*) : lectures fortes entre régions, aucune perte de données (RPO zéro)
 - Le mode par défaut, **MREC** (*eventual consistency*), garde le last writer wins

10 Sauvegarde et restauration

BACKUP ON-DEMAND

- Sauvegarde **complète** de la table, pour la rétention longue durée et la conformité
- Asynchrone, **aucune limite** de nombre, **n'affecte pas** le débit de la table

PITR (POINT-IN-TIME RECOVERY)

- Sauvegarde continue, rien à planifier
- Restaure la table à **n'importe quel moment des 35 derniers jours**
- Protège contre une écriture ou suppression accidentelle (ex. un script de test lancé sur la prod)
- ❖ **PIÈGE** Désactivé par défaut, à activer table par table
- ❖ **PIÈGE** Toute restauration crée une **nouvelle table**, elle n'écrase jamais la source
- Backup et restauration se déclenchent depuis la console, la CLI ou l'API

11 Conception single-table

partir des requêtes, puis dessiner les clés

DÉMARCHE

- Access patterns d'abord** : quelles requêtes l'application fait vraiment ? Les clés sont conçues pour les servir
- Single-table** : plusieurs types d'entités dans une seule table (ex. profil d'un user et ses transactions)
- Avantage : profil et transactions en un seul **Query**, sans jointure ni aller-retour multiple
- Coût : plus d'effort de design au départ

LES CLÉS

```
PK : USER#<user_id>
SK : PROFILE
SK : TXN#<timestamp ISO 8601>#<id>
```

- PK partagée : tout ce qui appartient à un user est dans la **même partition**
- Le préfixe **USER#** évite les collisions avec une autre entité qui réutiliserait le même ID

- PROFILE** : valeur fixe, il n'y en a qu'un par user
- TXN#...** : valeur variable, un user en accumule beaucoup

POURQUOI CET ORDRE DANS LA SK

- Timestamp ISO 8601 en premier** : l'ordre alphabétique égale l'ordre chronologique, donc un **Query** renvoie les items triés sans tri côté client

- ID à la fin** : garantit l'unicité même si deux items naissent à la même milliseconde (départage)

GSI : STATUS-INDEX

```
GSI partition key : status
GSI sort key      : created_at
```

- Permet « tous les items avec status = X », tous users confondus
- ❖ `created_at` trie chronologiquement dans chaque groupe
- Un GSI s'ajoute après coup, un LSI non
- ❖ **PRÉCISION** Voir la mise en garde sur les clés de GSI à faible cardinalité (branche 04)

SANS SCHÉMA, ET RELATIONS

- Un attribut s'ajoute plus tard avec `UpdateItem`, sans migration (contrairement à `ALTER TABLE`)
- Principe du minimum : pas de nom ni d'email dans le profil si un fournisseur d'identité les gère déjà

- Many-to-many** : un item de relation dont la SK sert de PK au GSI (et inversement), interrogeable dans les deux sens
- ❖ **PIÈGE** Hot partition aussi ici : un user extrêmement actif peut saturer sa partition

12 Débuguer une Lambda qui utilise DynamoDB

OÙ REGARDER

- Une Lambda n'a pas de terminal : ses `print` et exceptions n'apparaissent que dans **CloudWatch Logs**
- Un groupe de logs par fonction : `/aws/lambda/<nom-fonction>`, un log stream par invocation
- Démarche : erreur côté client (API Gateway cache tout derrière un « Internal server error » générique) → groupe de logs → stack trace → correction → redéploiement → test

PIÈGE 1 : DECIMAL NON SÉRIALISABLE

- `boto3.resource` renvoie les nombres en **Decimal** (Query, Scan, GetItem) pour garder la précision
- `json.dumps` plante : `TypeError: Object of type Decimal is not JSON serializable`

```
from decimal import Decimal

def decimal_default(obj):
    if isinstance(obj, Decimal):
        return int(obj) if obj % 1 == 0 else float(obj)
    raise TypeError(f'if {obj} n'est pas sérialisable')

json.dumps(data, default=decimal_default)
```

- Utile seulement dans les fonctions qui **lisent**. Une fonction qui n'écrit que (`put_item`) n'en a pas besoin, et l'ajouter sans la définir donne un **NameError**

PIÈGE 2 : IMPORT OUBLIÉ

- "Utiliser **Decimal** sans **from decimal import Decimal** donne `NameError: name 'Decimal' is not defined`, une erreur qui semble sans rapport avec le bug d'origine

- Réflexe : quand un code référence un type, vérifier son import

PIÈGE 3 : CORS AVEC LAMBDA PROXY

- La requête réussit côté serveur (200) mais le navigateur la bloque
- Le bloc **Cors** du template SAM ne génère les en-têtes que pour la réponse **OPTIONS** (preflight), pas pour la vraie réponse
- La Lambda doit renvoyer elle-même les en-têtes CORS, sur tous ses chemins, erreurs comprises

- ❖ **PIÈGE** Un preflight réussi ne prouve pas que la vraie requête passera

```
return {
    "statusCode": status_code,
    "headers": {
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Headers": "Content-Type",
        "Access-Control-Allow-Methods": "GET,POST,OPTIONS"
    },
    "body": json.dumps(data)
}
```

13 Hors de tes notes

à connaître pour le DVA-C02

LIRE SANS GASPILLER

- ❖ **PIÈGE** `Query` et `Scan` renvoient au max **1 MB** par appel : paginer avec `LastEvaluatedKey` → `ExclusiveStartKey`
- `KeyConditionExpression` : `PK =` et SK (`begins_with`, `between`, `<`, `>`)
- ❖ **FilterExpression** s'applique **après** la lecture : les RCU sont consommés pour tous les items lus, pas seulement ceux gardés
- `ProjectionExpression` réduit ce qui est renvoyé, pas le coût de lecture

ÉCRIRE EN SÉCURITÉ

- Condition Expression** (ex. `attribute_not_exists(PK)`) : si fausse, `ConditionalCheckFailedException`
- Compteur atomique** : `UpdateItem` avec `SET n = n + :inc`
- Verrouillage optimiste** : attribut `version` et condition `version = :v` à l'écriture, pour éviter les écrasements concurrents

BATCH ET TRANSACTIONS : LIMITES

- `BatchWriteItem` : **25** Put/Delete max (pas d'Update), **16 MB**
- `BatchGetItem` : **100** items max, **16 MB**
- Items refusés → `UnprocessedItems` à relancer avec **exponential backoff**
- Transaction : max **100** actions, **4 MB**
- ❖ **PIÈGE** Une opération transactionnelle coûte **2x** : 2 RCU par 4 KB lus, 2 WCU par 1 KB écrit

ERREURS ET CAPACITÉ

- ❖ `ProvisionedThroughputExceededException` (400) → retry avec **exponential backoff** (les SDK le font seuls) ; les erreurs 5xx aussi
- Auto Scaling** : ajuste les RCU/WCU provisionnés autour d'une utilisation cible (ex. 70 %)

TTL

- Un attribut **Number** en secondes epoch : une fois dépassé, l'item est supprimé
- Suppression en arrière-plan, pas instantanée (quelques heures à quelques jours)
- Gratuite (aucun WCU consommé), et visible dans les Streams

DAX

- Cache en **mémoire** devant DynamoDB, latence en **microsecondes**, compatible avec l'API DynamoDB
- Pour les lectures répétées et les points chauds

- ❖ **PIÈGE** Sert les lectures eventually consistant ; une lecture strongly consistant le contourne

DIVERS

- ❖ **PIÈGE** Item > 400 KB impossible : stocker le gros contenu dans **S3** et garder la référence dans DynamoDB
- PartiQL** : requêtes de style SQL sur DynamoDB
- Export vers S3** (depuis PITR), sans consommer de capacité de lecture

- Classes de table : **Standard** et **Standard-IA** (données rarement lues)

14 Glossaire des sigles

ACID	Atomicité, Cohérence, Isolation, Durabilité	CRUD	Create, Read, Update, Delete	PK	Partition Key (clé de partition, hash)
SK	Sort Key (clé de tri)	GSI	Global Secondary Index	LSI	Local Secondary Index
RCU	Read Capacity Unit (provisioned)	WCU	Write Capacity Unit (provisioned)	RRU	Read Request Unit (on-demand)
WRU	Write Request Unit (on-demand)	PITR	Point-in-time recovery (35 jours)	TTL	Time To Live (expiration automatique des items)
DAX	DynamoDB Accelerator (cache en mémoire)	MREC	Multi-Region Eventual Consistency (mode par défaut des Global Tables)	MRSC	Multi-Region Strong Consistency (mode optionnel)
SAM	Serverless Application Model	CORS	Cross-Origin Resource Sharing	ISO	ISO 8601 : date AAAA-MM-JJThh:mm:ssSZ, triable alphabétiquement

Les réflexes pour l'examen

- Chercher par un attribut qui n'est pas la clé → index secondaire (GSI, ou LSI si même PK avec une autre SK).
- Index à ajouter sur une table déjà créée → GSI. Un LSI se définit à la création.
- Lecture strongly consistant sur un index → LSI seulement. Un GSI est toujours eventually.
- Lecture à jour obligatoire → `ConsistentRead = true` (1 RCU / 4 KB). Défaut : eventually (0,5 RCU).
- Calcul RCU / WCU → arrondir la taille au supérieur (4 KB / 1 KB), eventually = ÷ 2.
- Transaction → coût x2, tout ou rien.
- Scan à éviter. Utiliser Query avec la PK, ou Query sur un index.
- FilterExpression ne réduit pas les RCU consommés.
- Hot partition → PK à forte cardinalité, trafic réparti.
- ProvisionedThroughputExceededException (400) → exponential backoff, Auto Scaling ou on-demand.
- Trafic imprévisible → on-demand. Stable et prévisible → provisioned.
- Réagir à chaque écriture → Streams + Lambda (24 h de rétention).
- Multi-région, multi-active → Global Tables (last writer wins).
- Erreur accidentelle sur la prod → PITR (35 jours), qui crée une nouvelle table.
- Expiration des données automatiquement → TTL.
- Cache en microsecondes devant DynamoDB → DAX.
- Item > 400 KB → S3 pour le contenu, référence dans DynamoDB.
- Écrasements concurrents → écriture conditionnelle (verrouillage optimiste).
- Lambda qui lit DynamoDB puis `json.dumps` → convertir les **Decimal**.
- CORS avec Lambda proxy → en-têtes dans la réponse de la Lambda, pas seulement dans le preflight.